# The SaaS Starting Point

---

## A practical framework for turning SaaS ideas into buildable systems

by Steve Westhoek

March 2026 v1.4.1

# Introduction

**AI builds code. Structure ships products.**

This document is for non-technical founders who want to stop guessing and turn a vague SaaS idea into something clear enough to validate, build, and sell.

It is not about:

- writing more prompts
- collecting more tools
- building faster just because AI made it possible

It is about:

- deciding what deserves software
- reducing rework
- avoiding expensive mistakes
- getting clear before you build the wrong thing

---

# Why Most SaaS Fails Before It Starts

Most founders do not fail because they cannot build.

They fail because they build too early.

AI made building easier.

It did **not** make decisions clearer.

That means you can now:

- build the wrong thing faster
- ship complexity sooner
- waste time more efficiently

## The real problem

Most SaaS fails before launch because founders start with:

- features
- tools
- stack
- UI
- automation

instead of:

- buyer
- pain
- outcome
- proof
- boundary

**One line to remember:**

**Software is not the reward for having an idea. Software is the consequence of a clear decision.**

> **WARNING**
>
> If you start with features before you define the buyer, pain, and outcome, AI will help you build confusion faster.

# What This Framework Helps You Decide

By the end of this framework, you should be able to define:

- **one buyer**
- **one pain**
- **one outcome**
- **one smallest proof**
- **one boundary**
- **one decision:** does this deserve software?

If you cannot answer those clearly, you are not ready to build yet.

That is not failure.

That is clarity.

---

# How to Use This Framework With AI

Do not use AI as a slot machine.

Use it in roles.

## Use AI as an interviewer

Ask AI to question you until your idea becomes specific.

Prompt direction:

- Ask me questions until my buyer, pain, and outcome are clear.
- Do not let me stay vague.

## Use AI as a critic

Ask AI to attack weak logic.

Prompt direction:

- Find where this idea is too broad, too vague, or too expensive to build.

## Use AI as a customer

Ask AI to simulate objections.

Prompt direction:

- Pretend you are the customer. What would make you ignore this?

## Use AI as a structurer

Ask AI to turn your answers into a clean decision document.

Prompt direction:

- Turn my answers into a one-page product decision brief.

**Rule:**

Do not let AI think instead of you.

Use AI to sharpen your thinking.

> **TIP**
>
> Use ChatGPT for reasoning first. Use your IDE agent second. Thinking is cheaper than rebuilding.

# Valuable Founder Tips

## Tip 1 — Start with what you already have

Most founders look outside too early.

Start with:

- your audience
- your niche
- your work
- your hobby
- your repeated frustrations
- your unfair insight

Ask yourself:

- What do I already understand well?
- What do people ask me for help with?
- What feels obvious to me, but confusing to others?

**Your own knowledge is often the best place to start.**

## Tip 2 — If you already have an audience, ask them

If you already have:

- followers
- clients
- subscribers
- viewers
- colleagues
- a niche network

then ask them:

- what frustrates them?
- what takes too long?
- what still feels manual?
- what breaks often?
- what do they wish existed?

Your audience is not just reach. It is market research with receipts.

## Tip 3 — Niche knowledge is a product advantage

If you work inside a niche, you may already know:

- the repetitive tasks
- the ugly workflows
- the spreadsheet chaos
- the missing integrations
- the manual work everybody accepts as normal

That is often where the best small SaaS ideas come from.

## Tip 4 — You do not need to be “the expert”

If you are even slightly ahead of someone else, you may already be useful to them.

If you are marginally better at:

- a profession
- a workflow
- a tool
- an industry process
- a hobby system

then through the eyes of a beginner, you are already an expert.

Do not wait until you feel like a professor.

Clarity beats credentials.

## Tip 5 — Keep the product small

Many useful SaaS products are just:

- one workflow
- one dashboard
- one automation
- one API bridge
- one clean wrapper around two broken systems

That is enough.

Do not ask: **What massive platform can I build?**

Ask: **What simple system would remove the most friction?**

## Tip 6 — Sometimes the product is just a connection

A lot of businesses already use:

- one tool for intake
- another for follow-up
- another for payments
- another for reporting

And the real problem is that they do not talk to each other.

That does not require a giant platform.

It may just require:

- one API connection
- one automation layer
- one unified screen
- one clean reporting view

That is still a real product.

> **SHORTCUT**
>
> If two existing systems already work, but the handoff between them is broken, that handoff may be the product.

# Step 1 — Define the Buyer

Who is this really for?

Not:

- founders
- businesses
- creators
- teams

Too vague.

## Better buyer definitions

- Portuguese accountants handling repetitive client requests
- real estate agents managing open-house follow-up
- coaches selling recurring digital offers
- small agencies juggling client onboarding manually

## Ask yourself

- Who already feels this pain?
- Who would immediately understand the value?
- Who can I picture using this?

## Output

**This is for [specific buyer] who struggles with [specific recurring problem].**

**COMMON MISTAKE**

Do not choose a buyer category so broad that your product has to become generic.

# Step 2 — Define the Pain

What hurts enough to matter?

A weak pain is annoying.

A strong pain costs:

- time
- money
- trust
- momentum

## Good pain signals

- repeated manual work
- losing leads
- inconsistent follow-up
- too many tools
- unclear workflow
- bad reporting
- fragile operations
- repetitive setup

## Ask yourself

- What is happening now that should not be happening?
- What is this problem costing?
- How often does it occur?
- Is this pain already real without my product?

## Output

**The pain is that [buyer] keeps experiencing [specific costly or frustrating problem].**

> **WARNING**
>
> If the pain only sounds useful after a long explanation, it is probably not sharp enough yet.

# Step 3 — Define the Outcome

What changes after they use this?

Do not describe the software.

Describe the result.

## Weak outcomes

- an AI dashboard
- a SaaS platform
- an automation tool

## Better outcomes

- leads get followed up consistently
- onboarding becomes repeatable
- recurring client work becomes structured
- one messy workflow becomes predictable

## Ask yourself

- What useful result does the buyer actually want?
- What changes in their day?
- What becomes easier, faster, safer, or clearer?

## Output

**The outcome is that [buyer] can now [specific result].**

**One line to remember:**

Features are not the outcome.

The change is the outcome.

# Step 4 — Define the Smallest Proof

What is the smallest version that proves this matters?

Not:

- the full dream
- the full company
- the full feature set

The smallest proof.

## This could be

- one dashboard
- one narrow feature
- one automation
- one guided workflow
- one manual-heavy MVP
- one API bridge

## Ask yourself

- What is the smallest useful version?
- What can stay manual?
- What would count as proof?
- What must be true before I expand this?

## Output

**The smallest proof is [minimum version] that proves [core value].**

**COMMON MISTAKE**

Most founders do not underbuild. They overbuild proof.

# Step 5 — Define the Boundary

What is in?

What is out?

What waits?

Without boundaries, everything sneaks in.

That is how founders create bloated products before they even have validation.

## Boundary questions

- What are we solving first?
- What are we explicitly not solving?
- What can stay manual?
- What would add complexity too early?
- What looks impressive but does not strengthen the core outcome?

## Output

### In scope

- ...
- ...
- ...

### Out of scope

- ...
- ...
- ...

> **WARNING**
>
> If everything feels important, nothing is defined well enough yet.

# Step 6 — Decide Whether This Deserves Software

Not every idea deserves software.

Some ideas deserve:

- a service first
- a workflow first
- a manual system first
- a content channel first
- a spreadsheet first
- nothing at all

That is normal.

## It deserves software when:

- the buyer is clear
- the pain is real
- the outcome matters
- the smallest proof is believable
- the boundary is disciplined
- software gives real leverage

**One line to remember:**

**Do not build because you can. Build because the decision is clear.**

> **REALITY CHECK**
>
> If the software is mostly there to make the founder feel productive, it probably does not deserve to exist yet.

# What “Ready to Build” Actually Means

You are ready to build when you can clearly answer:

- Who is this for?
- What pain does it solve?
- What outcome does it create?
- What is the first useful version?
- What is outside the boundary?
- Why does this deserve software?

If those are still fuzzy, you are still discovering.

Stay there until the logic is tight.

That is cheaper than building confusion.

## The test

If you had to explain your product in plain language to one real buyer, could you do it without drifting?

If not, keep refining.

# Preferred Toolset

Once the decision is clear, use a toolset that reduces chaos.

This is the tested ProChat recommendation.

## ChatGPT Plus

Use it as your main reasoning layer.

Why:

- fixed monthly cost
- strong for thinking
- strong for refining decisions
- better than burning credits in builder traps

## The lovable trap (aka the credit trap)

A lot of non-technical founders get seduced by credit-based builders.

It feels cheap at first.

It often is not.

Why:

- every unclear prompt costs you
- every retry costs you
- every change of mind costs you
- every messy idea becomes an expensive bill

That is the trap.

With ChatGPT Plus, your reasoning cost is fixed.

That changes your behavior. You can think clearly without feeling punished for iteration.

## Practical workflow

A simple workflow that works well:

- Use **ChatGPT** to reason and sharpen the plan
- Let ChatGPT help write the prompt properly
- Use **Codex inside your IDE** to execute
- Use stronger reasoning only when needed

This gives you a powerful system without runaway usage costs.

## ChatGPT

You can subscribe to ChatGPT Plus by clicking here.

## AntiGravity

Use it as your main IDE if you do not already know another one well.

Why:

- simple
- flexible
- tested
- reduces tool overwhelm

Pick one IDE and move.

Do not spend two weeks researching editors.

You can download Antigravity here.

## Wispr Flow

Talk instead of typing.

Why:

- faster input
- less fatigue
- better founder explanations
- more natural thinking

This is one of the easiest wins in the whole workflow.

If you speak your ideas instead of typing every thought, you can often work **2–5x faster** and explain much more clearly.

### Reader benefit

You can try Wispr Flow free for 30 days by using my link

## Vercel first

Start simple if you are new.

Why:

- easy first deployment
- fewer infra distractions
- good for learning the system
- good for an MVP

Use Vercel to get moving.

Do not block your first launch with server complexity.

# Hetzner later

Move when the product is real and you want more control.

Why:

- better economics
- more control
- stronger long-term setup
- better cost predictability once things get serious

### Reader benefit

You can start with free Hetzner credit through my link, which is usually enough to cover your first month or close to it depending on your setup.

**Rule:**

Do not spend weeks shopping for tools. Pick one coherent toolset and move.

# Where SaaSKit Fits

This framework is the **decision layer**.

SaaSKit is the **execution layer**.

This document helps you decide:

- who the buyer is
- what the pain is
- what the outcome is
- what the smallest proof is
- what belongs in scope
- whether it deserves software

SaaSKit helps you execute after that.

## What SaaSKit really removes

Without SaaSKit, once your decision is clear, you still have to think through:

- authentication
- billing
- app structure
- launch setup
- infrastructure decisions
- repeated configuration
- production plumbing

That is a big step.

SaaSKit removes that step.

It gives you:

- a production-ready foundation
- structured launch setup
- less repeated infrastructure work
- a faster path from clear idea to real product

## The real value of SaaSKit

SaaSKit does **not** replace the framework.

It removes the heavy setup layer **after** the framework.

That means:

- you do the thinking here
- you define what deserves software here
- then you install SaaSKit
- then you code your actual solution into the foundation

You skip the whole “what do I do with auth, billing, setup, structure, launch plumbing?” phase.

That is a huge head start.

## Correct order

**Clarity first. Execution second.**

Do not use SaaSKit to avoid thinking.

Use it once the thinking is done.

**Explore SaaSKit**

# Your Next Step

Before you build, write these down:

- one buyer
- one pain
- one outcome
- one smallest proof
- one boundary
- one decision

Then take those answers back into AI.

Ask AI to:

- challenge vague parts
- expose weak logic
- point out scope creep
- simulate buyer objections
- tighten the document

When the decision is clear, build.

### Ready to execute from clarity?

**Explore SaaSKit**

# Final Reminder

Do not build bigger.

Build clearer.

Do not add more.

Define better.

The goal is not just code.

The goal is not just another app.

The goal is a reliable micro-SaaS that solves one real problem clearly enough to launch, maintain, and sell.

**Most SaaS fails before it starts.**

**AI builds code. Structure ships products.**